Unit → 2

MY SQL

→ Database

- A collection of data in a format that can be easily accessed
- Software used to manage data is called as database management system

Databases

Relational	Non Relational
• data stored in tables	• data not stored in tables
• eg → my SQL, oracle	• eg → Mongo DB

→ Structured Query Language (SQL)

- Language used to interact with relational databases
- It's used to perform
  - C → Create
  - R → Read
  - U → Update
  - D → Delete

→ Tables

- columns store structure / schema (design)
- while rows stores individual data

→ CREATE DATABASE db-name;

- Used to create database
- not case sensitive (create database also work)

→ DROP DATABASE db-name;

- Used to drop database

→ USE DATABASE-NAME;

- enters database

→ Table creation

```
CREATE TABLE COLLEGE (
id INT PRIMARY KEY,  → can't be empty, is unique
name VARCHAR (50),
age INT
    );
```

→ entering data in Table

```
INSERT INTO COLLEGE (id, name)
VALUES (1, "AMAN", 26),
   (2, "MEOW", 21);
SELECT * FROM COLLEGE → prints the table
        └→ for all data
```

→ Data types in SQL

1. Char (string)
   - stores characters of fixed length
2. VARCHAR
   - stores string of variable length
3. BLOB
   - stores very large strings

- INT
- BIGINT
- TINYINT
- BIT → stores bits eg → BIT(2) → 01 10 00 11
- FLOAT
- DOUBLE
- DATE
- YEAR

→ signed & unsigned data types
TINYINT UNSIGNED → stores 0 to 255
TINY INT → stores -128 to 127

→ Types of SQL commands

1. DDL (Data defination language)
   - create, alter, rename, truncate, drop

2. DQL (Data query language)
   - select

3. DML (Data manipulation language)
   - insert, update, delete

4. DCL (Data control language)
   - grants & revokes permission to users

5. TCL (Transaction control language)
   - start, commit, rollback etc

→ CREATE DATABASE IF NOT EXISTS db_name;

→ DROP DATABASE IF EXISTS db_name;

- Used to create a database or drop it if it exists & vice versa.

→ SHOW DATABASES;

- Shows all available databases

→ SHOW TABLES;

- Shows all presint tables

→ Constraints in MY SQL

1. NOT NULL
   - col 1 int NOT NULL;
   - This col 1 can't be empty

2. UNIQUE
   - All values are different for different tuples
   - col 2 INT UNIQUE;

3. PRIMARY KEY
   - Can't be empty or redundant

```
CREATE TABLE temp (
id INT,
name VARCHAR (50),
age INT,
PRIMARY KEY (id)
);
```

Or

```
id INT PRIMARY KEY,
name VARCHAR (50),
:
:
```

4. FOREIGN KEY

```
CREATE TABLE temp (
cust_id INT,
FOREIGN KEY (cust_id) refrences customer(id)
);
```

cust_id → mayi table ki non key

customer(id) → purani table ki primary key

- Take primary key id from already existing table customer and puts its data into table temp's cust_id

5. DEFAULT

```
SALARY INT DEFAULT 25000;
```

- If value not entered then shows default value

6. CHECK

1.
```
CREATE TABLE CITY (
id INT PRIMARY KEY,
city VARCHAR (50),
age INT,
CONSTRAINT age_check CHECK (age>=18 AND
                            city ="DELHI")
);
```

OR

2.
```
CREATE TABLE NEW (
age INT CHECK (age >= 18)
);
```

- checks the constraints for eg here we can't enter age < 18 in this table

→ SELECT

- used to select any data from table

↳ SELECT name, id FROM STUDENT;
only shows id & name columns

↳ SELECT DISTINCT city FROM STUDENT;
only shows unique values
eg - if 2 students are from Delhi, it shows Delhi only once

↳ SELECT * FROM student WHERE marks > 80;
only shows data of marks where this condition > 80

→ Operators used with WHERE CLAUSE

1. AND
   - SELECT * FROM student WHERE marks > 80 AND city = "MUMBAI";
   - checks both conditions, true if both true
2. OR
   - checks, true if any one is true

3. BETWEEN
   - SELECT * FROM STUDENT WHERE marks BETWEEN 80 AND 90;

4. IN
   - SELECT * FROM student WHERE city IN ("Delhi")
   - Print data of students whose data in city column is Delhi

5. NOT IN

SELECT * FROM student WHERE city NOT IN "Delhi";

→ LIMIT CLAUSE

. SELECT * FROM student LIMIT 3;

. only prints first 3 data tuples from present in tuples of table

→ ORDER BY CLAUSE

. used to sort in ascending / descending order

SELECT * FROM student ORDER BY city ASC;

student → table name

city → column name

ASC → ascending order

→ Aggregate Functions

. Aggregate functions perform a calculation on a set of values & return single value

1. COUNT() → counts no of rows
2. SUM()
3. AVG()
4. MAX()
5. MIN()

SELECT AVG (MARKS) FROM STUDENTS;

SELECT MAX (marks) FROM STUDENT;

→ GROUP BY CLAUSE

. Groups rows that have same value into the summary rows

. It collects data from multiple records and the result in one or more column

```
SELECT city, count (name) from student
GROUP BY city;
```

Q SQL query to find avg marks in each city in ascending order

```
SELECT city, avg (marks)
FROM student
GROUP BY city
ORDER BY city;
```

→ HAVING CLAUSE

- Similar to WHERE i.e. applies some condition on rows
- Used when we want to apply any condition after grouping

```
SELECT count (name), city
FROM student
GROUP BY city
HAVING max (marks) > 90;
```

→ Table Related Queries

1. Update
- To update existing rows

```
UPDATE table - name
SET col1 = val1, col2 = val2
WHERE condition;
```

2. Delete

```
DELETE FROM table - name
WHERE condition;
```

→ Foreign Keys

Parent Table → Dept

id	name
101	Science
102	English
103	Hindi

child table → Teacher

id	name	dept_id (FK)
101	Adam	101
102	Bob	103
103	Casey	102
104	Donald	102

```
CREATE TABLE dept (
id int PRIMARY KEY,
name VARCHAR (50),
);
```

```
CREATE TABLE teacher (
id int PRIMARY KEY,
name VARCHAR (50),
dept_id INT,
FOREIGN KEY (dept_id) REFRENCES dept(id));
```

→ Cascading for foreign keys

1. ON DELETE CASCADE

When we create a foreign key using this option, it deletes the refirencing rows in child table when referenced row is deleted in the present table which has the primary key

2. ON UPDATE CASCADE

When we create a foreign key using UPDATE CASCADE, refirencing rows are

updated in the child table when the referenced row is updated in parent table

```
CREATE TABLE teacher (
id INT PRIMARY KEY,
name VARCHAR (50),
dept_id INT;
FOREIGN KEY (dept_id) REFERENCES dept (id)
ON UPDATE CASCADE
ON DELETE CASCADE
   );
```

} Changes in parent table also reflect in foreign key of child table

→ ALTER COMMANDS

1. ADD column

```
ALTER TABLE table_name
ADD COLUMN column_name data type;
```

2. DROP COLUMN

```
ALTER TABLE table_name
DROP COLUMN column_name;
```

3. RENAME TABLE

```
ALTER TABLE table_name
RENAME TO new_table_name;
```

4. Change column (Rename)

```
ALTER TABLE table_name
CHANGE COLUMN old_name new_name new_datatype
                    new_constraint;
```

5. MODIFY column (modify datatype / constraint)
ALTER TABLE table-name
MODIFY col-name new datatype new constraint

→ TRUNCATE command
DROP TABLE → deletes the table
TRUNCATE TABLE → deleted the data of table

TRUNCATE TABLE table_name;

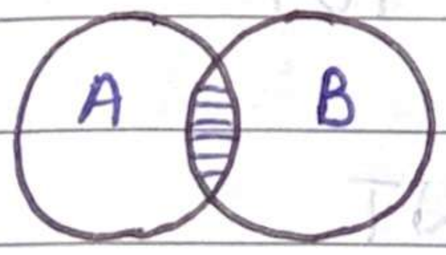
→ JOINS in SQL
• Join is used to combine rows from two or more tables based on a related column between them

employee
id	name
101	
102	

salary
id	salary
102	
103	

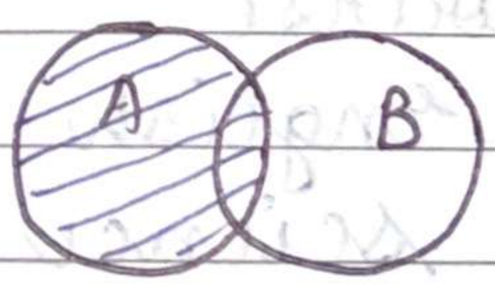
can fetch id, name, salary in a single table using these 2 tables

→ Types of Joins
1. Inner Join



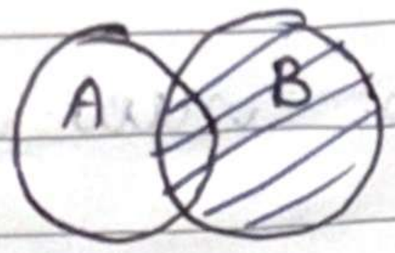
gives the common data between two tables A & B

2. Outer Joins
> Left Join



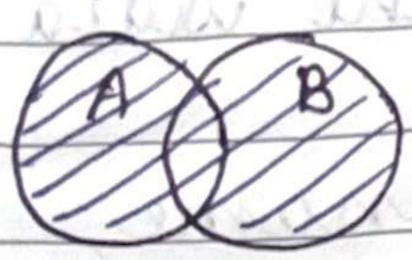
gives data of A along with A ∩ B

> Right Join



gives data of B along with A∩B

> Full Join

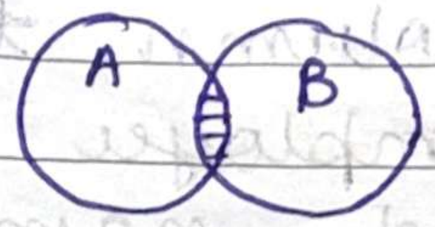


Returns complete data that is present in both the tables

→ INNER JOIN

- Returns the records that have matching values in both the tables

SELECT column(s)
FROM tableA
INNER JOIN table B
ON tableA.col-name = table B.col-name;



For eg:-

STUDENT

Student_ID	name
101	Adam
102	Bob
103	Casey

COURSE

student_id	course
102	English
105	math
103	Science
107	CSE

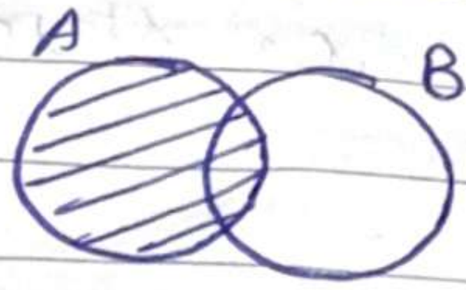
SELECT * FROM STUDENT
INNER JOIN COURSE
ON STUDENT.student_ID = COURSE.student_ID;

OUTPUT :-

student_ID	name	course
102	Bob	English
103	Casey	science

→ LEFT JOIN

Returns all records from the left table & matched records from right table



SELECT column(s)
FROM tableA
LEFT JOIN table B
ON tableA.col-name = tableB.col-name;

eg :-

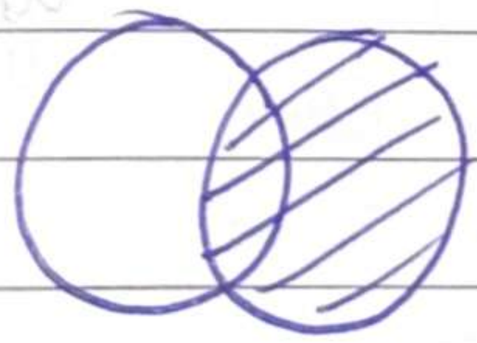
SELECT * FROM STUDENT AS S
LEFT JOIN COURSE AS C
ON S.student-ID = C.student-id;
OUTPUT

student-id	name	course
101	Adam	null
102	Bob	english
103	Casey	science

→ RIGHT JOIN

Returns all records from the right table and matched records from left table

SELECT column(s)
FROM table A
RIGHT JOIN table B
ON tableA.col-name = tableB.col-name;



eg :-

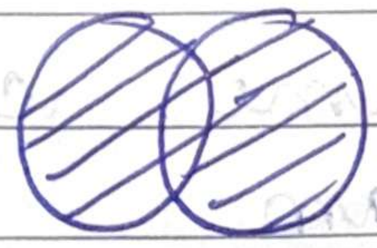
```
SELECT * FROM STUDENT AS S
RIGHT JOIN COURSE AS C
ON S.student_id = c.student_id;
```

Result :-

student_id	course	name
102	english	Bob
105	Math	null
103	Science	casey
107	CSE	null

→ FULL JOIN (UNION)

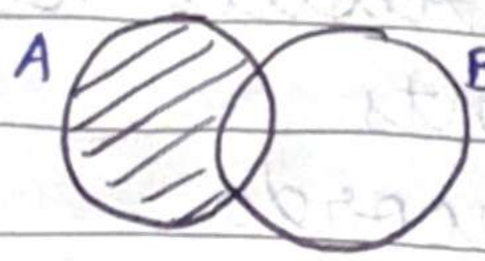
Returns full (all) records when there is a match in either of left / right tables



```
SELECT * FROM STUDENT as S1
LEFT JOIN COURSE as B
ON A.id = B.id
 UNION
 SELECT * FROM STUDENT as S
RIGHT JOIN COURSE as B
 ON S.id = B.id;
```

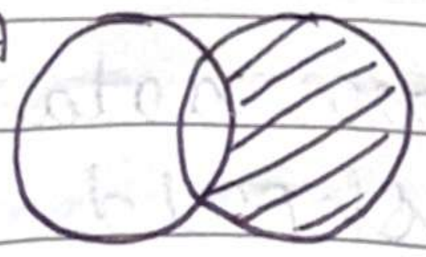
Syntax LEFT JOIN
UNION ⟶ Keyword in SQL
Syntax RIGHT JOIN ↳ by default only gives unique values

→ LEFT EXCLUSIVE JOIN




```
SELECT * FROM STUDENT AS A
LEFT JOIN COURSE AS B
ON A.id = B.id
WHERE B.id IS NULL;
```

→ RIGHT EXCLUSIVE JOIN




```
SELECT * FROM STUDENT AS A
RIGHT JOIN COURSE AS B
ON A.id = B.id
WHERE A.id IS NULL;
```

→ SELF JOIN

ID	NAME	MANAGER_ID
101	Adam	103
102	Bob	104
103	Casey	NULL
104	Donald	103

CASEY
↙ ↘
ADAM DONALD
↓
BOB

- It's used to join a table with itself like in this table manager id is given but manager name isn't given so we can use self join to find manager name

```
SELECT a.name as manager_name, b.name
FROM EMPLOYEE as A
JOIN EMPLOYEE as B
ON a.id = b.manager_id
```

→ UNION

- It's used to combine the result set of two or more SELECT statements
- It gives only UNIQUE records

→ conditions to use UNION

- every SELECT should have same number of columns
- columns must have similar data types
- columns in every select should be in same order

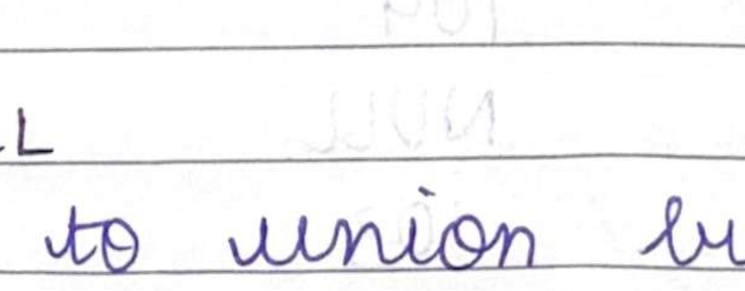
```
SELECT column(s) FROM table A
 UNION
SELECT column(s) FROM table B
```

→ UNION ALL

similar to union but it even returns the duplicate values

→ SQL sub queries

- A sub query / inner query / nested query is a query within another SQL query
- It involves two SELECT statements




```
SELECT column(s)
FROM table_name
WHERE col_name operator (subquery);
```

for eg :-

STUDENT

rollno	name	marks
101	Anil	78
102	Bhumika	93
103	Chetan	85
104	Dhruv	96
105	Emanuel	92
106	Farah	82

nested queries can be used to get the names of all students who scored more than class average.

```
SELECT name, marks
FROM STUDENT
WHERE marks > (SELECT AVG(marks)
               FROM STUDENT);
```

→ MY SQL VIEWS

- A view is a virtual temporary table based on result set of an SQL statement
- A view always shows up to date data The database engine re-creates the view everytime a user queries it

```
CREATE VIEW View1 AS
SELECT rollno, name FROM STUDENT;
SELECT * FROM View1;
```

→ Embedded SQL

Embedded SQL is a method of placing a database query statements directly inside the source code of a procedural host programming language (C/C++/FORTRAN)

While SQL is a declarative language optimised for manipulating sets of data, host programming languages provide procedural features like conditional branching, loops etc. Embedded SQL bridges this gap by executing database operations within traditional application logic

Source file (.pc / .sqc) → contains C code + Execute SQL statements

↓

Pre-compiler → like ORACLE, PRO*C, IBM SQL

↓ Replaces EXEC SQL with standard C API database function calls

Pure host code (.c)

↓

Host Language compiler → compiles C code & links database client libraries

↓

Executable binary

- To tell precompiler where host code stops and SQL code begins, statements are wrapped in special delimiters
- Host variables are declared in the host language that pass data into / recieve

data from SQL statements. In embedded SQL statements, host variable are prefixed with a colon (:). eg :-

→ starts SQL

EXEC SQL BEGIN DECLARE SECTION;
int emp-id = 101; [HOST → DB]
char emp-name [50]; [Database sends to host]
float emp-salary;

EXEC SQL END DECLARE SECTION;
EXEC SQL SELECT Name, salary
INTO :emp-name, :emp-salary
FROM EMPLOYEES
WHERE id = emp-id;
→ ends SQL

→ SQL returns data in multi row table set but procedural languages process data one row at a time. A cursor acts as an iterator over a query result set

- Declare cursor: Associate name with SELECT
- Open cursor: execute query & allocate memory
- Fetch: Retrieve current row into host variables & advance the pointer
- Close cursor: Release resources

eg :-

EXEC SQL DECLARE emp-cursor CURSOR FOR
SELECT name, salary FROM employees WHERE
department-id = 10;

EXEC SQL OPEN emp-cursor;

```
while (1) {
EXEC SQL FETCH emp_cursor INTO :emp_name,
                               :emp_salary;
printf ("Name : %.s, Salary : %.2f \n", emp_name,
        emp_salary);
}
```

```
EXEC SQL CLOSE emp_cursor;
```

→ NO SQL & MONGO DB

- NO SQL is a database system designed to handle flexible, unstructured data & high volume traffic without using traditional rigid SQL tables
- MONGO DB is world's most popular NO SQL database. Instead of storing data in rows and columns, it stores data in JSON like documents
- CORE Concepts :-

1. Document based: Each record is a JSON object. You store related data together in one place rather than splitting it across 5 joined tables
2. Flexible Schema : Columns aren't locked in. One document can have 3 fields, and another in same collection can have 10 fields - no migration scripts required
3. Built for Scale : Handles massive data growth across multiple servers automatically
4. Dveloper Friendly : Works natively with JSON making it seamless to use with modern languages like Node.JS, JavaScript, Python